OGC2026 Technical Report

Abstract. The challenge asks for a schedule and a physical layout at the same time: every block needs a bay, an entry time, and a pose that a vertically descending crane can actually reach. Our starting observation is that the objective $w_1 Z_1 + w_2 Z_2 + w_3 Z_3$ reads only the pair (bay, entry time)—coordinates and orientation appear nowhere in it. Geometry is therefore a feasibility constraint that decides how much scheduling freedom exists, not a term to be optimised directly. Since the share of the objective carried by tardiness ranges from 0% to 97.4% with nothing in between, we avoid density thresholds entirely: one portfolio of six operators competes for the budget on measured payoff per second, and every hot loop lives in one of two C++ extensions. Our main contributions are an exact contact upper bound that skips 40.8% of the candidate cells while provably returning the identical solution, a 10.5× faster conflict-graph build verified by an unchanged edge count, and a position-independent geometric predicate that removed a floating-point inconsistency from the packing test. The most instructive result is negative: that same verified 1.85× speedup made one instance 7.7% worse, because the operator schedule runs on a wall clock and a faster search therefore takes a different path rather than a longer one.

1 Introduction

1.1 Challenge Problem & Main Difficulties

Assigning hull blocks to bays over time is the spatial scheduling problem of shipbuilding, studied since the mid-1990s [2]. Three properties of the version posed here shaped everything we built.

The objective and the constraints read different variables. Tardiness, workload imbalance and preference are all functions of which bay a block occupies and when it enters; x, y and orientation appear in none of them. Geometry enters only through feasibility: a layout that cannot accommodate a block pushes its entry later, and that is the only channel through which packing quality becomes objective value. Treating tightness as an objective term is a modelling error we made early. Weighting contact more heavily does not buy a better schedule, it buys a denser layout whose scheduling consequences are unmeasured.

The crane makes the packing three-dimensional. A block is lowered vertically, so a placement is legal only if every layer of the descending block clears everything present along the whole descent. Two neighbours occupying the same floor cells but standing one and two layers high ration space completely differently: the scarce resource is height, not occupancy, which is why the descent test is layer-by-layer.

Instances are not on a spectrum. On the final-round instances the tardiness share of the objective runs from 0% to 97.4% with nothing in between, and the preliminary training set splits the same way: Z_1 is exactly zero on twenty of its forty instances and strictly positive on the other twenty (Table 1). Where Z_1 vanishes the entry times are free and the problem is an assignment dominated by preference; where the yard is saturated they are the whole objective. A hand-drawn threshold has to guess where that switch lies and the distribution gives no evidence about it, so we built a scheduler that measures which operator is paying rather than one that classifies the instance.

1.2 Contributions

- A *gate-free operator portfolio*: six operators compete for the budget on realised objective gain per second, with no density threshold anywhere; slice sizing is a separate question answered by completion.
- An *exact contact upper bound* skipping 40.8% of candidate cells while provably returning the identical solution, verified per cell rather than by comparing run outcomes.
- A 10.5× faster *conflict-graph build*, accepted on an unchanged edge count rather than a stopwatch, over a shared memo made safe by compare-and-exchange.
- *Offset-relative geometry*: the packing predicate became position-independent, fixing a disagreement we had misdiagnosed as hashing.
- A negative result more useful than any of these: an exact, verified 1.85× speedup that made one instance 7.7% worse (Section 2.3).

2 Algorithm Description

2.1 Overall Algorithm Framework

Four stages, and no preprocessing worth the name. *Initial solution:* each of four independent worker processes seeds a pool with a cheap always-feasible sequential schedule — the one guarantee that the algorithm never returns nothing, and the reason every other fallback path could be deleted. *Improvement:* each worker then runs an operator loop until its budget is exhausted, differing from the others in random seed and in its starting offset into six diversification axes, so the four explore different trajectories over the same operators. *Selection:* the best full objective across workers is taken, and the entry point reserves $\min(0.2T, 40\,\mathrm{s})$ of the limit for a final preference-repair pass over that winner. *Validation:* feasibility is enforced inside the operators and re-checked outside them — one acceptance function re-runs the provided checker on anything that beats the incumbent and returns $+\infty$ if it fails. An operator may be wrong about feasibility; a wrong answer cannot be accepted.

Six operators are available: a contact-beam construction; an evolutionary re-growth with crossover and path-relinking over (bay, dispatch-order) anchors,

decoded by the same beam; a workload-balance repair; a preference repair; a CP-SAT bay reassignment; and the exact bay repacking of Section 3. Each receives one probe to establish a rate, after which the budget goes to whichever has the highest realised objective gain per second, with 15% exploration so a slow starter can recover. The arrangement is an adaptive large neighbourhood search [4] whose adaptive weights score realised objective gain per second rather than a tally of accepted moves.

Selection and sizing are different questions. Slice size is deliberately not driven by the selection signal. Sizing on payoff death-spirals: a construction returning a good solution that merely fails to beat the incumbent scores zero gain, shrinks, and can then no longer finish at all — fourteen calls returning four solutions while the objective moved from 48,840 to 56,841. Size is a completion question instead: an operator that returned nothing was starved and is given more, one that returned anything fitted, and a repair pass is given what it used.

2.2 Key Ideas that Succeeded

Contact beam, and why its contact measure is flat. The construction is a filtered beam search over dispatch order [3], ranking survivors on the exact objective rather than a priority index. Each state places the next block at the position minimising a weighted sum of negative contact, a positional sweep term and a preference penalty, and survivors are completed by a rollout. Contact plays the role touching perimeter plays in two-dimensional packing, extended to a descent the crane must clear.

Contact admits two measures. A per-layer one describes the geometry better, since a block beside a shorter neighbour earns nothing for the layers facing open air, and it is implemented. The submitted configuration nevertheless uses the flat measure, because the bound below is derived against a flat occupancy prefix sum, which does not dominate per-layer occupancy: the two cannot both be on, and the bound is worth 1.09–1.85× in scanned cells at an identical placement digest. This is a constraint we accepted rather than a property we wanted; with a per-layer bound the choice would likely go the other way, and that derivation is left undone.

An exact contact upper bound. Profiling showed 86% of scored cells were evaluated after the best cell of that scan had already been found. Bounding the positional term fired zero times in 112.9 million cells: the score is contact-dominated by more than an order of magnitude (ranges of ≈ 2.3 against ≈ 60), so a bound knowing only the sweep coordinate is never competitive.

Bounding the contact itself inverts this. Contact counts, for each occupied footprint cell, its four neighbours: one for a neighbour outside the bay, nothing for one inside the same footprint, and its weight for an occupied one. Every cell that can contribute therefore lies inside the footprint bounding box dilated by one and counts at most once per direction, giving

$$\mathrm{ct}(i_x, i_y) \;\le\; W(i_x, i_y) \;+\; 4\, w_{\max} \left| \mathcal{O} \cap B^{+1}(i_x, i_y) \right|, \tag{1}$$

where W counts cell–direction pairs pointing out of the bay, $\mathcal{O}$ is the occupied set and B^{+1} the dilated box. An occupancy prefix sum, built once per (bay, time window) at the same asymptotic cost as the occupancy map it accompanies, answers the rectangle in four reads, and a cell whose best possible score cannot beat the incumbent is skipped without any geometric test. The bound is applied only where it is proved, which is what excludes the per-layer variant.

A cliff at the bottom of the beam, and the axis that came out of it. The beam checked its deadline at each level and returned nothing when it passed, so every worker fell back to the sequential floor. The failure was not gradual: on a 300-block instance the answer was 4.02×10^9 at 110 s against 9.69×10^7 at 130 s, a factor of 42, and 143 on a 250-block one, at a threshold that moves with instance size while the hidden limits are undisclosed. The repair needed no new machinery, since the beam already completes a partial state by rollout to rank survivors: on expiry it completes the incumbent rather than discarding it.

That exposes a knob rather than a patch: the fraction of its slice the beam claims before handing the remainder to the rollout. Sweeping it down improves large instances monotonically (-24.3% to -11.9% at one tenth) and harms small ones monotonically, since the beam finishes anyway there and a low aim only narrows it ($+25.09\%$ at 150 blocks). We did not resolve that by testing instance size. The workers are already a portfolio whose minimum is the answer, so half run the low aim and half the high one, each group carrying the instances the other loses, and nothing is gated. Over 37 paired final instances this is 31 better against 6 worse, median -3.44%, sign test $p < 0.0001$, splitting 17–2 on the large group and 14–4 on the small. Our prediction that the small group would pay was wrong: true of a constant, false of an axis. A portfolio does not need every member to be good, it needs them to fail on different instances.

Replicating the two small instances that lost by more than 10% shrank both to a median of $+3.2\%$, and one arm of the first swings 29.3% against itself, wider than either gap between the arms. A single paired draw at 150 blocks is worth its sign, not its magnitude, which is why the table above is a sign count.

A conflict-graph build that watches its own clock. The repacking operator begins with an $O(n_{\mathrm{col}}^2)$ conflict graph that cannot be interrupted. Two columns conflict only while both are present, so sorting by entry time and breaking the inner loop once the later entry passes the earlier exit skips 83% of pairs without visiting them; the fields the filter reads moved into flat arrays. Neither change can alter which pairs conflict, so the edge count had to come out identical, and that, not the clock, was the acceptance test. The same configuration now builds in 10.7–14.2 s against about 90 s before. The build also projects its own completion every 256 rows and steps down to a cheaper configuration it can finish rather than losing the call.

The rows are independent, so the loop is finally parallel. What is not independent is where the results go: each thread keeps its own edge list, and the memo on the conflict predicate is shared. Sharing it naively is not merely racy but silently wrong, because the table is open-addressed and two threads inserting different

keys can both claim one empty slot, the loser then writing its verdict into the winner's entry. A slot is therefore claimed by compare-and-exchange and only by that; relaxed ordering suffices because the value is a pure function of the key. A private memo per thread gave almost all the parallel gain back (1.10×), since reuse is where this loop's speed lives, while sharing it correctly reaches 1.43× on four cores at an edge count of 133,863,098 in every arm.

We then failed to spend the headroom. Widening the 40-outsider candidate cap had once cost 331 s of a 180 s budget and now costs 179 s, but the wider list scored 3 better against 4 worse, median +0.17%: the evidence for wanting it came from two instances and did not survive a larger sample.

Pricing a seat, not a block. The operator hands the solver a list of candidate placements, a *column* being one block at one orientation, position and entry time, and asks for the maximum-weight conflict-free selection. The weight was attached to the *block*. Since the objective reads only (bay, entry time) and one call concerns one bay, the entry variant is exactly the granularity at which a block's alternatives differ: an on-time column and a ten-days-late column into a preferred bay carried the same number, so the solver took whichever seated more easily.

Every attempt to express the trade from outside failed for the same reason. Discounting a block's weight by the cost of its late seat undervalues it when it takes the on-time one, and at the break-even delay, the only principled place to put the offer, the discount equals the entire gain: the weights of precisely the high-regret blocks collapse to the floor. Across the final instances that form is indistinguishable from never offering a late window at all, at 9 better against 21 worse, median +0.00%.

Attaching the weight to the entry variant removes the workaround rather than tuning it. Each offered window is priced by the only term an entry time can move,

$$p(w) \;=\; g \;-\; w_1\big(\max(0,\, e_w - d) - \max(0,\, e_0 - d)\big), \tag{2}$$

with g the gain of the move, e_w the exit under window w, e_0 the current exit and d the due date. It is exact in both directions, so a window *earlier* than the block's current entry prices above the base and the solver can be paid for pulling a block forward, which a per-block weight cannot represent at all. On 39 final instances the priced form is 20 better against 9 worse (sign test $p \approx 0.03$, mean -0.74%) and does no harm where no late window can open.

Offset-relative geometry. Memoising the conflict predicate produced 36 extra edges. We assumed key aliasing; dumping the disagreements showed the predicate itself was position-dependent through floating-point rounding. Comparing origin outlines plus an offset makes the verdict independent of absolute position, which fixed the memo by fixing the predicate.

2.3 Key Ideas that Failed

A wall-clock schedule turns a speedup into a different answer. This is the result we would most like to pass on. Having verified that the contact upper bound is exact, that it returns bit-identical placements at fixed work, and that it is 1.85×, 1.66× and 1.09× faster on three instances of very different density, we enabled it and one instance got 7.7% worse.

The bound is not at fault, and neither is anything else in the search. The operator loop decides what to run next, and for how long, from elapsed seconds. A faster scan fits more calls into the same budget, which shifts the diversification axis rotation, the contents of the solution pool, and the repacking operator's call counter and hence its random seed. That operator is then handed a different incumbent, and on this instance exactly one incumbent, worth 107,195, ever leads it to 83,095, which is in turn the only route to the best solution we have recorded there. Across the whole day every run reaching that solution had 83,095 in its log and every run that did not, did not. With the bound off we reproduce it three times in four; with it on, never in two.

The lesson generalises beyond this bound. Making a time-budgeted portfolio faster does not make it search longer along the same path, it makes it take a different path, and the new one need not be better. We priced that per instance, as the competition scores, and shipped with the bound disabled: −7.7% against +2.2% is a losing trade.

A better incumbent can repack worse. Incumbents worth 97,570, 100,685 and 107,195 yielded repacking improvements of 9,580, 13,510 and 24,100: better incumbents repack worse. A tight solution has no slack left and this operator eats slack, so its result is a function of the incumbent's layout rather than its objective — yet the roster hands it the best-scoring solution every time.

Two optimisations that were simply wrong. A free-column bitset for the packing search was proved to give identical results and then measured 1.6–1.9× *slower*: its saving scales with columns per block, its cost with edges, and we had assumed the wrong line was hot. We also twice predicted the contact bound's benefit from its firing rate (1.3% and 3.4% of cells) and were wrong in the same direction both times. Most cells are rejected by a cheap bitmap probe, while the cells a bound removes are exactly those that would have reached the exact geometric test. Cell counts are not seconds.

2.4 Further Improvement Plan

The first item is to remove the wall clock from the schedule's decisions. We localised where it enters: with the deadline lifted the construction returns identical placements with and without the bound, but under the deadline it receives the two diverge. The divergence begins in the construction's stopping rule rather than the operator loop, and the repair belongs there; the packing solver already takes an iteration cap for exactly this reason. Once the schedule depends on

work done rather than seconds elapsed, the contact bound can be re-enabled and contribute its speed instead of a different trajectory.

Second, the repacking operator should sample the solution pool rather than always taking its best member, since its yield depends on slack rather than quality; the re-growth operator already samples the pool, so the mechanism exists.

Third, that operator's cost model deserves the scrutiny its search has had. It declines a configuration whose predicted build will not fit the budget, and the prediction charges a fixed parsing cost to a quadratic term and mixes two different column counts. Both errors are squared, and both push towards declining work the budget could afford.

3 Implementation Details

Python 3.12 orchestrates and two C++17 extensions built with `pybind11` and OpenMP hold every hot loop. CP-SAT serves one operator and the algorithm degrades to the remaining five if it is unavailable. Python holds only control flow, the scheduler, the operator definitions and the acceptance test; every geometric predicate, every candidate scan and both search engines are C++. The construction parallelises over beam states and the portfolio over worker processes, never nested.

The repacking operator. This operator lifts every block out of the most contested bay, adds the outside blocks that might want to enter it, and hands the set to a maximum-weight set-packing solver over space–time columns weighted in objective units. Destroying a bay and rebuilding it is ruin and recreate [5] with an exact recreate step, and the selection is a maximum-weight independent set searched by the swap-based scheme of Andrade et al. [1]. A call has two phases of different character: an uninterruptible build and a deadline-honouring search. The solver takes a total-time bound and enforces it against its own measured build cost rather than a predicted one; an earlier version subtracted a predicted build from the search budget as well, double-charging it. The configuration ladder is expressed as fractions of each instance's own entry-time ceiling, so window widths come from the instance rather than being fitted to it.

Verification instrumentation. The engines carry counters for cells visited, exact tests, cheap rejections and cells skipped by the bound, plus a soundness mode that scores every cell the bound wanted to skip and counts those that would have won: zero out of 45.9, 11.6 and 9.3 million on three instances of very different density. An earlier bound had passed an identity check that was meaningless because the branch never executed, so we stopped accepting outcome comparisons as evidence.

Table 1. Preliminary-round training set, all forty instances at 60 s. n blocks, m bays.

Inst.	n	m	Z_1	Z_2	Z_3	Obj.	Inst.	n	m	Z_1	Z_2	Z_3	Obj.
1	100	2	0	157	2	1,499	21	100	3	8	2,032	2,693	530,934
2	100	3	0	144	15	3,690	22	100	2	2	2,391	1,748	733,039
3	100	3	0	1,542	186	43,320	23	100	2	83	1,237	1,284	1,390,856
4	100	2	0	2,278	0	15,946	24	100	3	1	4,553	661	234,398
5	150	3	0	1,654	228	45,778	25	100	2	317	397	1,792	247,676
6	150	3	0	2,346	173	42,372	26	150	3	523	1,197	4,150	7,604,038
7	150	3	0	2,335	239	52,195	27	150	2	1,695	845	4,422	24,369,925
8	150	2	0	2,413	8	11,252	28	150	3	17	2,371	1,892	801,374
9	200	3	0	2,047	271	50,885	29	150	3	1	7,860	969	327,613
10	200	4	0	2,225	400	68,775	30	150	2	123	573	1,442	1,930,651
11	200	4	0	923	96	19,229	31	200	4	121	1,025	10,436	4,402,780
12	200	4	0	2,205	387	66,906	32	200	3	415	2,080	2,065	2,632,595
13	250	4	0	3,905	297	59,026	33	200	3	955	1,824	4,977	7,131,775
14	250	4	0	1,106	350	52,080	34	200	4	44	2,051	1,256	830,457
15	250	4	0	4,276	100	34,680	35	200	3	21	2,032	1,485	512,903
16	250	4	0	3,294	139	34,957	36	250	4	18	4,500	6,496	100,954
17	300	4	0	6,563	238	57,906	37	250	3	614	4,506	3,154	3,956,886
18	300	4	0	4,188	198	43,086	38	250	3	2,607	1,987	4,164	36,012,305
19	300	4	0	4,604	301	58,449	39	250	3	500	1,218	5,711	7,528,022
20	300	5	0	2,704	652	97,724	40	250	4	2,629	3,957	7,947	1,860,811

4 Computational Results

Table 1 reports all forty preliminary-round training instances at a uniform 60 s budget, decomposed into the three objective components; a uniform budget is the only comparable one, since the hidden limits vary and are not disclosed. All forty solutions pass the provided feasibility checker, and the split noted above is visible directly: Z_1 is zero on instances 1–20 and positive on 21–40.

Across submissions. Table 3 gives the objective each of our three submissions returned on the eight final-round instances. The honest reading is that it does not separate the changes from the noise: every per-instance move is smaller than the spread the algorithm produces against itself — three replicates of one unchanged build returned 1,075,322, 831,362 and 923,531, a range of 29.3%, against a largest move here of 7.2% — and both shipping steps bundled two changes. What the table records is direction: five of eight improved from the first submission to the third, and the total is flat.

Per-seat pricing, on the final practice set. The change described in Section 2.2 was judged on all forty final practice instances, paired, at 180 s, against two controls: no late window offered at all, and the late window offered under the old per-block weight. Judging by sign count over the set rather than instance by instance is deliberate. A single instance answers the same question with a spread

Table 2. Final-round training set, all forty instances at 60 s, same build. Different instances from Table 1: w_1 spans 333 to 13,559 against the preliminary 667 to 29,630, and w_3 reaches 800 against 600.

Inst.	n	m	Z_1	Z_2	Z_3	Obj.	Inst.	n	m	Z_1	Z_2	Z_3	Obj.
1	150	3	17	5,858	799	610,313	12	300	5	11	15,458	5,215	149,496
2	250	3	4,412	1,774	8,179	60,059,142	13	300	2	9,207	2,120	7,538	67,415,589
3	200	4	167	6,823	7,746	4,732,990	14	250	4	1,617	2,467	5,171	680,545
4	250	4	296	1,705	3,165	2,682,038	15	250	3	1,294	2,802	5,947	12,200,902
5	150	2	2,122	182	2,068	7,901,464	16	300	5	173	1,203	4,813	3,513,468
6	250	4	344	4,078	5,904	5,392,174	17	250	3	876	631	7,845	5,278,256
7	150	3	53	715	2,145	1,033,404	18	200	3	2,402	585	3,815	10,434,705
8	200	4	203	2,383	2,199	129,355	19	200	3	1,129	2,936	8,434	17,589,029
9	200	3	323	5,960	6,575	6,110,361	20	250	3	1,169	2,984	8,968	9,162,795
10	150	2	305	7,364	2,176	4,531,221	21	150	2	11	8,212	2,104	1,004,687
11	250	4	417	8,236	7,304	7,534,737							

of about 3%, and unchanged code shifted 8.4% between two runs an hour apart on this machine, so no single pair can resolve an effect of this size.

The priced form is 20–9 and does no harm where no window can open, the condition we had set in advance for enabling it by default. The effect shrank as the sample grew, from -1.61% at twelve instances to -0.74% at thirty-nine: the first twelve had been chosen as the most favourable.

Two cautions belong with these numbers rather than after them. Run-to-run spread on the densest instance is about 112,000 on unchanged code, so smaller differences are noise. The sparsest is not stable at all: its objective is a minimum over the repacking operator's attempts within a run, that operator's improvement varies from 6,590 to 31,315 with the incumbent it is handed, and the value we quote appeared in four runs of seven — a mode, not a reproducible figure.

Speed, measured at identical work. A fixed-time A/B cannot evaluate an accelerated search: both arms consume the same budget, so the faster one legitimately reaches a different solution. We replay one construction call with identical arguments and no deadline, comparing a digest over every returned placement. On a sparse, a mid-density and a dense instance the bound skipped 40.8%, 3.4% and 1.3% of candidate cells, none unsound, for 1.85×, 1.66× and 1.09× at an identical digest in all three.

The conflict-graph build was accepted only after its edge count was confirmed unchanged on every configuration tested, for a 10.5× speedup in isolation.

5 Team Members' Contributions

The rules require anonymity, so members are identified by role. **Member A** led the algorithmic work: problem analysis, the decision to treat geometry as

Table 3. Objective returned on the eight final-round instances by each of our three submissions. Between the first and the second we shipped per-seat window pricing and the parallel conflict-graph build; between the second and the third, the beam salvage of Section 2.2 and the split-aim worker portfolio.

Instance	Submission 1	Submission 2	Submission 3	1 to 3
1	3,068,862	3,328,237	2,847,060	−7.2%
2	19,829,534	20,337,489	20,063,787	+1.2%
3	5,886,589	5,867,652	5,613,271	−4.6%
4	4,421,375	4,283,430	4,681,440	+5.9%
5	6,308,099	6,104,015	6,148,480	−2.5%
6	1,024,046	1,053,770	968,674	−5.4%
7	16,385,030	16,285,457	16,310,765	−0.5%
8	17,079,881	16,023,014	17,347,930	+1.6%
Total	74,003,416	73,283,064	73,981,407	−0.0%

Table 4. Late entry windows on the final-round training set: 39 instances paired at 180 s against the same solver with no late window offered, split by whether the instance's weights permit one at all.

	Better	Worse	Same	Mean
trade available (35 instances)				
per-block weight (previous)	9	21	5	+0.62%
per-seat price	20	9	6	−0.74%
no trade available (4 instances)				
per-seat price	2	1	1	−0.53%

a feasibility constraint, the measured-payoff portfolio, the contact bound and its proof, the conflict-graph and geometry optimisations, and the diagnosis of Section 2.3. **Member B** owned the codebase and the experimental process: version management, packaging, the harnesses and the discipline of running every comparison on an unloaded machine, the sweeps, and verification that reported numbers matched the logs behind them. Both contributed to result analysis and to this report.

6 AI Use Declaration

One tool was used, substantially: Anthropic Claude, through its command-line coding interface, as an interactive coding and analysis assistant throughout development. It implemented and refactored the two C++ extensions and the Python scheduler from specifications settled in discussion; derived the contact bound of Section 2.2 and its validity conditions; wrote the measurement harnesses, including the per-cell soundness check and the identical-work replay; ran the experiments; and diagnosed defects, among them the position-dependence of

the geometric predicate. It drafted this report. Algorithmic direction, prioritisation and every decision to accept or reject a change remained with the authors, and every number here was produced by executing the submitted code on the challenge instances; none was generated or estimated by a language model.

References

1. Andrade, D.V., Resende, M.G.C., Werneck, R.F.: Fast local search for the maximum independent set problem. J. Heuristics **18**(4), 525–547 (2012)
2. Lee, K.J., Lee, J.K., Choi, S.Y.: A spatial scheduling system and its application to shipbuilding: DAS-CURVE. Expert Syst. Appl. **10**(3–4), 311–324 (1996)
3. Ow, P.S., Morton, T.E.: Filtered beam search in scheduling. Int. J. Prod. Res. **26**(1), 35–62 (1988)
4. Ropke, S., Pisinger, D.: An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. Transp. Sci. **40**(4), 455–472 (2006)
5. Schrimpf, G., Schneider, J., Stamm-Wilbrandt, H., Dueck, G.: Record breaking optimization results using the ruin and recreate principle. J. Comput. Phys. **159**(2), 139–171 (2000)